1. AMI (Amazon Machine Image)

What it is:

An Amazon Machine Image (AMI) is a pre-configured template that contains the information required to launch an EC2 instance. It includes the operating system, application server, applications, and configuration settings. Think of it like a snapshot of a virtual server.

How to create or customize one:

To create a custom AMI, you typically start an EC2 instance using an existing AMI (like Amazon Linux or Ubuntu). After configuring the instance by installing software, setting configurations, and securing the environment, you can create an AMI from that instance. This AMI can then be reused to launch identical instances.

AWS-provided vs. Custom AMIs:

AWS provides a wide range of ready-to-use AMIs, including standard Linux and Windows OS images, which are maintained and updated by AWS. Custom AMIs are created by users to preserve specific configurations, environments, or software stacks tailored to their needs. AWS AMIs are ideal for general use, while custom AMIs are better suited for replicating consistent internal setups or deployments.

---

2. Instance Types

Instance Families and Use Cases:

EC2 instance types are grouped into families based on their compute, memory, and storage capabilities.

- The T3 family is known as burstable performance instances. These are designed for workloads that don't use the full CPU all the time but occasionally need to burst.
- The M5 family represents general-purpose instances. These offer a balanced ratio of compute, memory, and networking for most workloads.
- The C5 family is compute-optimized and is suitable for high-performance computing tasks, batch processing, or scientific modeling.

Each family suits different use cases. For example, T3 is suitable for lightweight websites or development environments, M5 for application servers and small databases, and C5 for data analytics or machine learning tasks.

Quick Use Case Mapping

- **T3** → Budget apps with spikes
- **M5** → General production workloads
- **C5** → High-CPU tasks like video encoding, compute-heavy apps
- **R5** → RAM-heavy tasks like Redis, SAP HANA
- **I3** → Fast local storage needs
- **P3** → AI/ML model training
- **G4** → ML inference, graphics rendering
- **H1** → Big data storage and analytics

Comparison Example:

Feature	T3.medium	M5.large
vCPUs	2	2
RAM	4 GB	8 GB
CPU	20% baseline, bursts to 100%	100% always available
Cost/month	~$30	~$70
Best for	Variable workloads	Consistent performance

Burstable (T3) vs General-Purpose (M5):

Burstable instances like T3 use a credit-based system. They accrue CPU credits when idle and spend them during bursts. They are cost-efficient for unpredictable or variable workloads.

General-purpose instances like M5 provide consistent and sustained performance. They are ideal when you expect constant workload and performance requirements.

---

3. Key Pairs (.pem, .ppk files)

Purpose and Use:

Key pairs are used to securely access EC2 instances. When you launch an instance, you select a key pair. The private key (.pem for Linux or .ppk for Windows with tools like PuTTY) is stored locally, while AWS stores the public key.

When you SSH (for Linux) or use RDP (for Windows) to connect to your instance, the private key verifies your identity. Without the matching key, access is denied.

Best Practices:

- Store private keys securely and never share them.
- Use strong file permissions to protect them on your system.
- Rotate keys periodically and delete old ones.
- Avoid embedding keys inside images or instances.

---

4. Security Groups

Configuration of Inbound and Outbound Rules:

Security groups act as virtual firewalls for your instances. They control inbound (incoming) and outbound (outgoing) traffic.

For example, to allow Remote Desktop access to a Windows instance, you add an inbound rule for port number used by that service. To allow SSH access for Linux, you add a rule for the respective port.

Outbound rules are usually open by default, allowing all traffic from your instance to the internet. You can restrict these as needed.

Inbound Rules:

- Control traffic *coming into* your instance (from the internet or other servers).
- Example:
  - Allow RDP (port 3389) for Windows remote access.
  - Allow SSH (port 22) for Linux login.
  - Allow HTTP (port 80) for web traffic.

Without an inbound rule for a port, no one can access that service.

Outbound Rules:

- Control traffic *going out* from your instance (to the internet or other servers).
- Example:
  - Allow all outbound traffic (default in AWS).
  - Restrict access to certain IPs or ports for tighter security.

Without outbound rules, your instance can't connect out (e.g., to download updates).

Difference Between Security Groups and NACLs:

Security groups are associated with instances and are stateful. If you allow traffic in one direction, the return traffic is automatically allowed.

Network Access Control Lists (NACLs) are associated with subnets and are stateless. They require explicit rules for both inbound and outbound traffic. NACLs provide an additional layer of security at the subnet level.

---

5. EBS (Elastic Block Storage)

GP2 vs. GP3:

Both are general-purpose EBS volume types used for attaching block-level storage to EC2 instances.

GP2 volumes provide a baseline performance with bursts based on volume size. As the size increases, so does performance. GP3 volumes, on the other hand, offer consistent performance regardless of size and allow you to configure performance parameters independently from storage size. GP3 is generally cheaper and more flexible than GP2.

Modifying or Resizing Volumes:

EBS volumes can be resized or modified while the instance is running. You can increase the size, change the type (e.g., from GP2 to GP3), or adjust performance settings.

After modifying the volume, you typically also need to resize the filesystem inside the instance so it can use the additional space. No data is lost during resizing if done properly.

---

1. AMI (Amazon Machine Image)

Steps to Create a Custom AMI from an Instance:

1. Launch an EC2 instance using any base AMI (e.g., Amazon Linux).
2. Install software, make configurations.
3. Go to **EC2 Dashboard > Instances**.
4. Select your instance, click **Actions > Image and templates > Create image**.
5. Fill in:
    - Image name and description
    - Add any EBS volumes
6. Click **Create image**.

CLI Command to Create AMI:

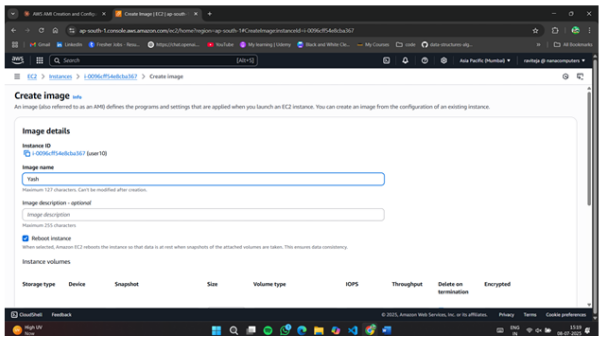
Command: aws ec2 create-image --instance-id i-0abcdef1234567890 --name "MyCustomAMI" --no-reboot

Feature	AWS-Provided AMI	Custom AMI
Source	By AWS	By user
Configuration	Default OS only	Pre-installed apps and configs
Use Case	Base setups	Replication or scaling setups
Updates	Automatically managed	Must be manually updated

Creating an AMI from an existing instance:

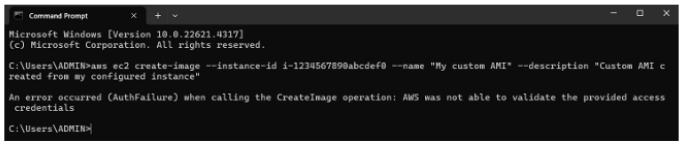
Method 1: Using AWS Management Console:

- Navigate to the EC2 console
- Select your running instance
- Choose "Actions" → "Image and templates" → "Create image"
- Provide image name and description
- Configure additional settings (reboot behavior, block device mappings)
- Click "Create image"



Method 2: Using AWS CLI

```
aws ec2 create-image --instance-id i-1234567890abcdef0 --name "My custom AMI" --description "Custom
AMI created from my configured instance"
```



---

2. Instance Types

Common EC2 Families:

Family	Type	Description	Use Case
T3	Burstable	Low-cost with burstable CPU	Development, low-traffic apps
M5	General	Balanced compute-memory-network	Web servers, databases
C5	Compute	High-performance processors	HPC, ML inference, analytics
R5	Memory	High memory-to-CPU ratio	In-memory DBs like Redis
I3	Storage	High disk throughput	NoSQL DBs, data warehousing
P4	GPU	Powerful GPU support	Deep learning, video processing

---

3. Key Pairs (.pem / .ppk files)

Steps to Create a Key Pair:

- **Console:**
  1. Go to **EC2 Dashboard > Key Pairs**.
  2. Click **Create Key Pair**.
  3. Choose:
     - Name
     - Format: `.pem` for Linux, `.ppk` for PuTTY (Windows)
  4. Save the key securely.
- **CLI:**
  aws ec2 create-key-pair --key-name MyKeyPair --query 'KeyMaterial' --output text > MyKeyPair.pem chmod 400 MyKeyPair.pem

Convert `.pem` to `.ppk` for Windows:

- Use **PuTTYgen**
  1. Load `.pem`
  2. Save private key as `.ppk`

4. Security Groups

Steps to Configure:

1. Go to **EC2 > Security Groups**.
2. Create a new security group or select an existing one.
3. Add **Inbound Rule** for SSH or RDP:

Protocol	Port	Source	Use Case
SSH	22	Your IP (recommended)	Linux access
RDP	3389	Your IP (recommended)	Windows remote access
HTTP	80	Anywhere (0.0.0.0/0)	Public web server

4. Add **Outbound Rule** (default: all traffic allowed).

Security Group vs NACLs Table:

Feature	Security Groups	NACLs
Applied to	Instances	Subnets
Stateful	Yes	No
Rule direction	Inbound + outbound	Inbound and outbound separate
Rules evaluated as	Allow only	Allow and deny

---

5. EBS (Elastic Block Store)

GP2 vs GP3 Volume Comparison:

Feature	GP2	GP3
Performance	Scales with volume size	Provisioned IOPS independent
Max IOPS	16000	16000
Cost	Higher	Lower
Flexibility	Less	More (IOPS and throughput control)

Resizing an EBS Volume:

- **Console:**
  1. Go to **EC2 > Volumes**.
  2. Select the volume, click **Modify Volume**.
  3. Enter new size or change volume type.
  4. Click **Modify**.

- **CLI:**
  aws ec2 modify-volume --volume-id vol-123abc456 --size 100 --volume-type gp3

Tasks:

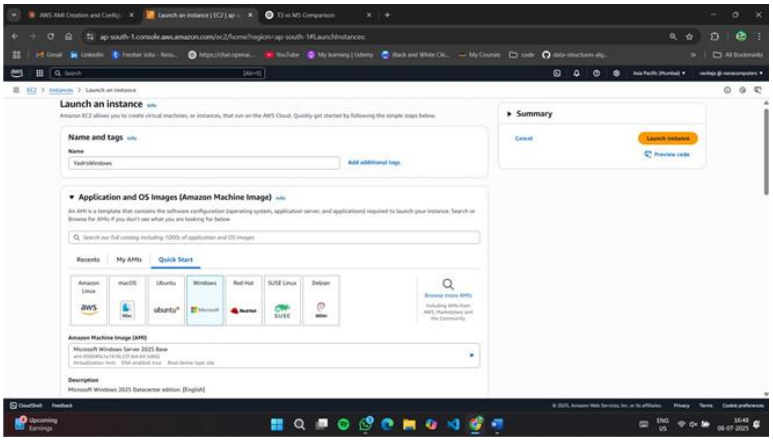
*** Launch a Windows Server Instance (T3.micro) and Connect via RDP**

Steps:

1. **Login to AWS Console > EC2**
2. **Launch Instance:**
   - Name: `WinServerTest`
   - OS: Microsoft Windows Server (e.g., 2022 Base)
   - Instance Type: `t3.micro` (eligible under free tier)
   - Key Pair: Create or choose existing `.pem` file
   - Network: Choose default VPC
   - Security Group: Add inbound rule for RDP (port 3389)
   - Storage: Default 30 GB
3. **Launch**
4. After the instance is in the "running" state, select it → Click **Connect** > **RDP Client**
5. **Download remote desktop file** and **decrypt password** using your private key file (`.pem`)
6. **Open RDP**, paste the password, and connect

Findings:

- RDP requires **open port 3389** in the security group.
- `t3.micro` performs decently for UI-based operations but not ideal for heavy tasks.
-



***Shared Responsibility Model (EC2 vs. S3 vs. RDS):**

Layer	EC2 (IaaS)	S3 (Storage)	RDS (PaaS)
Hardware	AWS	AWS	AWS
Network	AWS	AWS	AWS
Virtualization	AWS	N/A	AWS
OS	User	N/A	AWS
Patching (OS)	User	N/A	AWS
Application Security	User	User	Limited (User + AWS)
Encryption	User-managed	AWS-managed or User-managed	AWS-managed or User-managed
Access Control	User (IAM policies, SGs)	User (IAM, bucket policies)	User (IAM + DB users/roles)
Backup & Restore	User	AWS + User	AWS (Snapshots, auto backups)

Conclusion:

- **IaaS (EC2)** gives users full control of the environment, thus more responsibility.
- **PaaS (RDS)** offloads patching, backup, and some availability responsibilities to AWS.
- **S3 (Storage)** is highly managed; most tasks involve setting correct access and lifecycle policies.

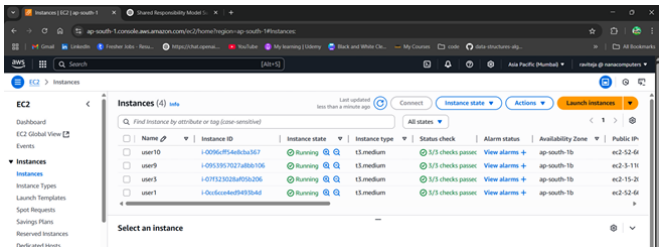
Regions & Availability Zones

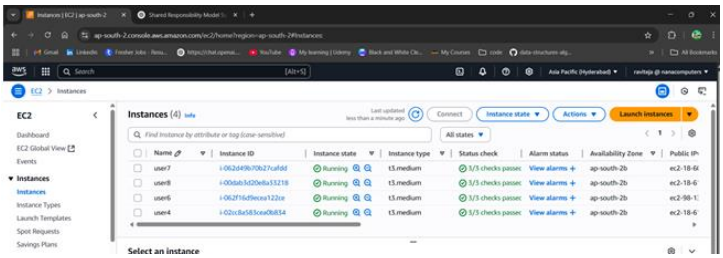
Steps:

1. In AWS Console, switch region to `us-east-1`.
2. Launch another `t3.micro` instance (same AMI or different).
3. Compare latency, IP addresses, and data accessibility with your `ap-south-1` instance.

Findings:

- Regions are completely isolated.
- Each region has different **availability zones** (AZs), useful for fault-tolerance.
- **AMI IDs are region-specific.**
- Latency to `ap-south-1` is lower when accessing from India.





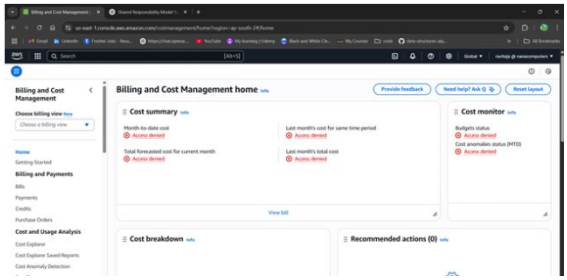
Cost Monitoring

Using AWS Billing Dashboard:

1. Go to **Billing > Dashboard**
   - View current month's usage
   - Check Free Tier usage
   - View service-level cost breakdown
2. **Setup Billing Alerts:**
   - Go to **CloudWatch > Alarms**
   - Create a billing alarm
   - Select metric: `Billing > Total Estimated Charges`
   - Set threshold (e.g., INR 10 or USD 0.1)
   - Configure SNS email notification

Findings:

- CloudWatch doesn't show billing metrics by default — **must enable in us-east-1 region only.**
- Billing alerts help prevent unexpected charges, especially from idle resources or storage.



Troubleshooting EC2

Simulated Error: RDP Not Working

Issue Simulated:

- Removed RDP (3389) rule from Security Group

Troubleshooting:

1. **Instance Status Checks:**
   - Go to **EC2 > Instances**
   - Check for "2/2 status checks passed"
   - If failed: issue is internal or system-level
2. **Check Security Group:**
   - Inbound Rules → Missing RDP (3389)? Add it
   - Source should be `My IP` or `0.0.0.0/0` (for testing)
3. **Elastic IP Attached?**
   - Confirm public IP hasn't changed after reboot (or assign static Elastic IP)
4. **Network ACLs (if modified):**
   - Confirm ACLs allow inbound/outbound traffic on port 3389

Findings:

- Most RDP issues come from misconfigured **security groups** or incorrect **key pair** use.
- Instances with failed status checks often require a reboot or replacement.

Summary of Actions & Tools Used:

Task	Tool/Service Used
EC2 deployment	EC2 Dashboard + Key Pair
RDP Connection	Windows RDP + Decrypted password
Shared Responsibility Review	AWS Docs + Role mapping
Region comparison	EC2 console (us-east-1 vs ap-south-1)
Cost monitoring & alerts	Billing Dashboard + CloudWatch
Troubleshooting	EC2 Monitoring tab + Security Group

Advantages vs Disadvantages of Key EC2 Concepts:

Topic	Advantages	Disadvantages
EC2 Instance Deployment	- Full control of OS, apps, network- Scalable & flexible- Pay-as-you-go	- Requires manual setup & patching- RDP/SSH setup may fail if security is misconfigured
Windows on T3.micro	- Free tier eligible- Good for light workloads- Easy GUI access with RDP	- Limited CPU burst- Not ideal for memory or compute-intensive tasks
Key Pairs (.pem/.ppk)	- Secure SSH/RDP login- Easily generated and managed	- Loss of private key = loss of access- Needs strict permission settings
Security Groups	- Easy to manage firewall rules- Stateful traffic filtering	- Only allow rules (no deny)- Misconfiguration causes blocked access
Shared Responsibility Model	- Clear boundaries of who manages what- Offloads infrastructure management in S3/RDS	- Can confuse beginners- Users might ignore their security responsibilities on EC2
AMI (Custom vs AWS)	- Custom AMIs speed up repeated deployments- AWS AMIs are trusted and updated	- Custom AMIs need manual updates- AMIs are region-specific and not globally accessible
Regions & AZs	- Fault tolerance through AZs- Global deployment flexibility	- AMIs and resources are region-bound- Latency may vary across regions
Cost Monitoring	- Prevent surprise bills- Billing alarms via CloudWatch	- Detailed cost analysis requires setup- CloudWatch billing only available in one region
EC2 Troubleshooting	- Built-in status checks- Logs, metrics, and alerts help debug	- Some issues need deep network/system knowledge- Mistakes in SG/NACLs can block everything